

Relatório de Evolução — Sprint 3

ChargeGrid AI | EV Challenge 2026 | GoodWe × FIAP

7.1 O que mudou

Na Sprint 2 o chatbot funcionava assim: o usuário mandava uma mensagem, o código montava uma lista com o histórico e passava tudo para o modelo manualmente. Funcionava, mas tinha limitações claras — a memória dependia do código não esquecer de atualizar a lista, e os guardrails existiam só no texto do system prompt.

Na Sprint 3 refatoramos o núcleo usando LangGraph. Agora o fluxo passa por um grafo com dois nós: um de guardrail e um de agente. A memória é gerenciada automaticamente pelo MemorySaver com thread_id. A proteção contra prompt injection virou um nó separado no grafo, que roda antes de qualquer chamada ao modelo.

Também migramos da tentativa de uso do Gemini (Google) para a API da OpenAI com o modelo GPT-4o mini, por problemas de compatibilidade dos SDKs do Google com o Python 3.14.

7.2 Decisões técnicas

Por que LangGraph?

Na Sprint 2 já usávamos LangChain. O LangGraph é construído em cima dele, então a migração foi mais simples do que aprender algo do zero. O principal ganho foi poder montar um grafo de estados com nós separados — um para guardrail e outro para o agente — em vez de tudo no mesmo bloco de código.

Por que MemorySaver com thread_id?

Na Sprint 2 o histórico era uma lista Python passada manualmente a cada chamada. Com MemorySaver o LangGraph gerencia o estado por thread_id automaticamente. Cada sessão tem seu próprio histórico isolado, sem risco de misturar contextos entre usuários diferentes.

Por que GPT-4o mini?

Tentamos usar o Gemini (Google) mas os SDKs disponíveis tinham incompatibilidade com Python 3.14. Migramos para a OpenAI usando a chave de API fornecida pelo professor. O GPT-4o não estava disponível nessa chave de projeto, então usamos o GPT-4o mini, que respondeu corretamente em todos os testes.

Limitações encontradas:

- O MemorySaver não salva em disco — se o programa fechar, a memória some
 - A lista de triggers do guardrail precisa ser atualizada manualmente
 - O LangGraph tem mais conceitos para aprender do que o LangChain puro
 - Não foi possível comparar dois modelos distintos com a chave disponível; a comparação foi feita por variação de temperature
-

7.3 Antes e depois

	Sprint 2	Sprint 3
Framework	LangChain direto	LangGraph
Modelo	Gemini 1.5 Flash (planejado)	GPT-4o mini (OpenAI)
Memória	Lista manual por turno	MemorySaver automático por <code>thread_id</code>
Guardrail de injection	Só no system prompt	Nó separado no grafo
Sessões isoladas	Não	Sim
Testes funcionais	5/5	5/5
Testes de memória	Não tinha	3/3
Testes de segurança	1	5
Comparação de modelos	1 configuração	2 configurações (temperature 0.3 vs 0.7)

7.4 Problemas encontrados

Problema 1 — SDK do Google incompatível com Python 3.14

Tentamos usar o `google-generativeai` e depois o `google.genai`, mas os dois deram erro com Python 3.14. O `google-generativeai` está descontinuado e o `google.genai` novo ainda não tinha suporte estável para essa versão do Python.

Alternativas consideradas:

- Instalar uma versão mais antiga do Python
- Usar o Google Colab para contornar o problema
- Trocar para outro provedor de API

Solução adotada: migramos para a API da OpenAI (GPT-4o mini), que funcionou sem problemas no Python 3.14.

Problema 2 — Campo `blocked` sendo sobrescrito entre turnos

O LangGraph usa `add_messages` para acumular o histórico automaticamente. Quando criamos o campo `blocked` para sinalizar que o guardrail bloqueou uma mensagem, o valor estava sendo sobrescrito errado entre um turno e outro.

Alternativas consideradas:

- Criar dois grafos separados para mensagens bloqueadas e normais
- Usar `conditional_edges` para rotear diferente dependendo do resultado
- Tratar o campo dentro do próprio nó do agente

Solução adotada: o nó do agente verifica `state.get("blocked")` internamente e retorna a recusa direto, sem chamar o modelo. O campo é resetado a cada nova chamada para não afetar o próximo turno.

7.5 Divisão da equipe

Nome	RM	Turma	Responsabilidade
Gabriel Souza	571583	CCPQ	Desenvolvimento do agente LangGraph, implementação do guardrail e integração com a API OpenAI
Rafael Sá	569223	CCPQ	Documentação dos casos de teste, comparação entre modelos e elaboração do relatorio_modelos.md
João Melo	571116	CCPQ	Elaboração do relatório de evolução, README do repositório e organização dos entregáveis